

EverCurrent Technical Interview Questions — AI

LLM Behavior

1. How do you handle LLMs losing context in long-running or multi-step workflows?

I first decide what information really needs to survive between steps, such as source IDs, permissions, earlier decisions, and the user's current intent. I prefer to keep that information as structured state outside the model instead of depending on an increasingly long chat history. Each step should receive only the context it needs, return a small validated result, and save important decisions so the workflow can be retried without asking the model to reconstruct everything from scratch.

In this Daily Digest project, retrieval, grouping, ranking, and ACL checks are deterministic. The summarizer only sees the final authorized Top-K stories, along with source IDs and relationship evidence, and its output is checked against the source content before being cached. If I extended this into a longer workflow, I would add clearer step schemas, versioned state, and replayable checkpoints before simply increasing the model context window.

Retrieval

2. How do you address cases where vector similarity returns many nearly identical results with small but critical differences?

I treat similarity as a way to find candidates, not as proof that two engineering messages mean the same thing. In hardware work, a small difference like 24V vs. 48V, a different revision, part number, date, or test condition can completely change the meaning. So I would first filter by ACL and metadata, then combine semantic candidates with keyword and exact-identifier matches, and compare those critical fields before grouping messages together.

For this MVP, the dataset is small, so I used deterministic TF-IDF plus exact-match retrieval instead of adding embeddings just for complexity. The relationship logic then separates duplicates, explicit replacements, conflicts, and unresolved differences. For example, M001 and M002 are comparable measurements with different values, while M001 and M003 were taken at different RPMs and should not be treated as a direct contradiction. At production scale, I would likely use embeddings to improve recall, but I would still keep identifier-level checks and source traceability.

Evaluation

3. How do you evaluate LLM generated content/summaries?

I separate **selection quality** from **summary quality**. A summary can sound good but still be useless if the system selected the wrong information. For selection, I use human-reviewed persona × phase relevance labels, report Precision@5 for each scenario and the macro average, and check whether rankings change in a reasonable way across personas and project phases.

For summary quality, I check whether important identifiers such as numbers, units, revisions, dates, and part IDs are actually supported by the authorized source messages. I also test invalid or unsupported outputs and check that conflicts or superseded values are still attributed correctly. If validation fails, the demo falls back to a deterministic summary. I would still use human review for things like factual attribution, usefulness, and missing information, because an identifier check alone cannot prove that the full meaning of a summary is correct.

System Design

4. When would you use function calling vs MCP in production?

I would start with the scope of the system. If one application has a small and known set of operations, typed functions or direct function calling are usually simpler to test, authorize, and maintain. That is enough for this MVP because the pipeline is local and the available operations are fixed. Adding MCP here would mostly add another layer without improving the actual digest experience.

I would consider MCP when the same governed capabilities need to be discovered and reused across multiple model clients or tools. For example, if EverCurrent later wanted its engineering knowledge or digest capabilities to be shared across several assistants, MCP could provide a standard way to expose those tools and schemas. I would still keep important logic such as ranking and ACL on the server side, because MCP does not replace access control, source validation, or audit logging.

5. How would you handle Slack API failures?

I would handle different failure types differently. Rate limits should follow Retry-After; temporary network or server failures should use bounded exponential backoff with jitter; and authentication or permission errors should stop retrying and trigger an authorization or operator workflow instead.

I would also make ingestion idempotent using Slack event or message IDs, keep a cursor or checkpoint, and use a bounded polling pass to recover missed events after an outage. Most importantly, a partial failure should not silently look like a complete digest. The UI should show data freshness and source coverage, and if the data is stale I would either show the last known digest with a clear warning or avoid showing a misleading update. This take-home uses a fixed synthetic snapshot, so real Slack OAuth, Events API handling, and retry infrastructure are intentionally not implemented; those would be the next reliability layer for a production version.